

VisionInspect AI

Manufacturing Defect Detection & Quality Inspection System

Milestone 3 Report

PatchCore Model Inference & Serving Layer

Prepared by: Sarang Soni

Team: 4

Program: Infosys Springboard Internship 7.0

Date: August 2026

Table of Contents

1. Work Summary
 - 1.1 Modules Completed
 - 1.2 Tools and Technologies Used
2. Model Inference Module (predict.py)
 - 2.1 Checkpoint Resolution Logic
 - 2.2 Inference Workflow
3. Inference API (app.py)
 - 3.1 Endpoints
 - 3.2 Verdict & Severity Logic
4. Batch Validation (batch_validate.py)
5. Repository Reorganization
6. Results & Screenshots
7. Observations
8. Conclusion

1. Work Summary

1.1 Modules Completed

Building on the 15 trained PatchCore models from Milestone 2, Milestone 3 focused on turning those trained checkpoints into an actually usable inference and serving layer:

- Built a standalone inference module (`predict.py`) that loads any trained category's checkpoint and runs anomaly detection on a new image.
- Implemented checkpoint resolution logic that correctly locates the latest trained model version for a category, handling Anomalib's versioned output folder structure (`latest/`, `v0/`, `v1/`, ...).
- Generated anomaly heatmap overlays for each prediction, visually localizing the detected defect region on the original image.
- Wrapped the inference logic in a FastAPI service (`app.py`) exposing health-check, category-listing, and prediction endpoints over HTTP.
- Added a severity-bucketing and pass/fail verdict layer on top of the raw anomaly score.
- Built a batch validation script (`batch_validate.py`) that runs inference across all 15 categories and produces a results CSV.
- Reorganized the repository into `training/` and `inference/` subfolders for clearer project structure.

1.2 Tools and Technologies Used

Category	Technology
Programming Language	Python
Inference Framework	Anomalib (PatchCore)
API Framework	FastAPI + Uvicorn
Image Processing	OpenCV, NumPy
Version Control	Git & GitHub

2. Model Inference Module (`predict.py`)

`predict.py` is a standalone script that loads a trained PatchCore checkpoint for a given category and runs inference on a single new image, independent of the training pipeline.

2.1 Checkpoint Resolution Logic

Anomalib saves each training run into a versioned output folder structure. To guarantee the newest trained model is always used, `find_checkpoint()` follows a priority order:

- Prefer the `latest/` folder if it exists and contains a checkpoint.
- Otherwise, scan all `vN/` version folders and use the highest-numbered one that contains a checkpoint.
- As a last resort, fall back to a plain directory walk of the category folder.

```
def find_checkpoint(category):
    category_dir = os.path.join(RESULTS_DIR, "Patchcore", "MVTecAD", category)
    latest_dir = os.path.join(category_dir, "latest")
    if os.path.isdir(latest_dir):
        ckpt = _find_ckpt_under(latest_dir)
        if ckpt:
            return ckpt
    # fall back to highest-numbered vN folder, then a full directory walk
```

2.2 Inference Workflow

Given a category and an image path, `predict_image()` loads the PatchCore model architecture, restores the trained weights from the resolved checkpoint, and runs a single forward pass through Anomalib's `Engine.predict()`. The result includes a predicted label (Normal / Defective), a raw anomaly score, and a pixel-level anomaly map, which is overlaid on the original image and saved as a heatmap.

```
python predict.py --category bottle --image "../dataset/.../bottle/test/broken_large\000.png"
```

3. Inference API (app.py)

`app.py` wraps `predict.py` in a FastAPI service so the trained models can be called over HTTP by a frontend or another backend service, instead of requiring a manual command-line run for every prediction.

3.1 Endpoints

Endpoint	Method	Purpose
/health	GET	Confirms the server and GPU are available
/categories	GET	Lists which of the 15 categories have a trained checkpoint ready
/predict	POST	Accepts an image + category, returns verdict, score, severity, and a heatmap URL

3.2 Verdict & Severity Logic

Raw anomaly scores are converted into a human-readable severity level and a binary pass/fail verdict, so downstream consumers do not need to interpret raw floating-point scores directly:

```
def get_severity(pred_label, pred_score):
    if pred_label == "Normal": return "None"
    if pred_score >= 0.85: return "Critical"
    if pred_score >= 0.70: return "High"
    if pred_score >= 0.50: return "Medium"
    return "Low"

verdict = "FAIL" if pred_label == "Defective" else "PASS"
```

Note: severity thresholds are a starting point based on the score separation observed during initial testing (0.94 for a defective sample vs. 0.21 for a normal sample on the bottle category) and are expected to be refined once validated across more sample images per category.

4. Batch Validation (batch_validate.py)

To confirm all 15 trained models work correctly end-to-end (not just the ones manually spot-checked), batch_validate.py automatically selects one normal image and one defective image per category, runs both through predict_image(), and records whether each prediction matched the expected label. Results are saved to a CSV file for easy review.

- Iterates through all 15 trained categories automatically.
- Selects one sample from test/good/ and one from the first available defect subfolder per category.
- Logs predicted label, score, and a YES/NO correctness flag for each sample.
- Continues past any single category's failure rather than stopping the whole run.
- Saves a full results table to batch_validation_results.csv.

5. Repository Reorganization

The project structure was reorganized to clearly separate model training work from the new inference/serving work, making it easier for a reviewer to navigate the repository:

```
src/  
├── training/  
│   ├── train_patchcore.py  
│   └── patchcore_training_log.json  
├── inference/  
│   ├── predict.py  
│   ├── app.py  
│   ├── batch_validate.py  
│   └── batch_validation_results.csv  
└── notebooks/
```

Files with existing git history (train_patchcore.py, patchcore_training_log.json) were moved using git mv to preserve their commit history rather than appearing as newly added files.

6. Results & Screenshots

The inference pipeline was validated on the bottle category using one known-defective test image and one known-normal test image, confirming correct discrimination between the two classes.

```
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
▶ Tips for seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable Litlogger, which logs metrics and artifacts automatically to the Lightning experiments platform.
You are using a CUDA device ("NVIDIA GeForce RTX 3050 6GB Laptop GPU") that has Tensor Cores. To properly utilize them, you should set 'torch.set_float32_matmul_precision('medium' | 'high')', which will trade-off precision for performance. For more details, read https://pytorch.org/docs/stable/generated/torch.set_float32_matmul_precision.html#torch.set_float32_matmul_precision

Restoring states from the checkpoint path at D:\Internship_Project\VisionInspectAI\src\results\Patchcore\WrecAD\bottle\weights\lightning\model.ckpt
D:\Internship_Project\venv\lib\site-packages\lightning\pytorch\callbacks\model_checkpoint.py:566: The dirpath has changed from 'D:\Internship_Project\VisionInspectAI\src\results\Patchcore\WrecAD\bottle\weights\lightning' to 'D:\Internship_Project\VisionInspectAI\src\predictions\Patchcore\latest\weights\lightning', therefore 'best_score', 'kth_best_model_path', 'kth_value', 'last_model_path' and 'best_k_models' won't be reloaded. Only 'best_model_path' will be reloaded.
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
Loaded model weights from the checkpoint at D:\Internship_Project\VisionInspectAI\src\results\Patchcore\WrecAD\bottle\weights\lightning\model.ckpt
D:\Internship_Project\venv\lib\site-packages\lightning\pytorch\trainer\connectors\data_connector.py:434: The 'predict_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the 'num_workers' argument to 'num_workers=11' in the 'DataLoader' to improve performance.
Predicting: 100% 1/1 0.000s < 0.000s 2.001s
[predict] heatmap saved to: D:\Internship_Project\VisionInspectAI\src\predictions\bottle_000_heatmap.png
[predict] RESULT: Defective (score=0.9399)

=====
PREDICTION RESULT
=====
category: bottle
image path: ..\dataset\wrec_anomaly_detection\bottle\test\broken_large\000.png
pred_label: defective
pred_score: 0.9399
heatmap path: D:\Internship_Project\VisionInspectAI\src\predictions\bottle_000_heatmap.png
(venv) PS D:\Internship_Project\VisionInspectAI\src >
```

Fig 6.1: Console output — inference correctly running on GPU (CUDA) and returning a Defective verdict (score = 0.9399) for a known-defective bottle image.

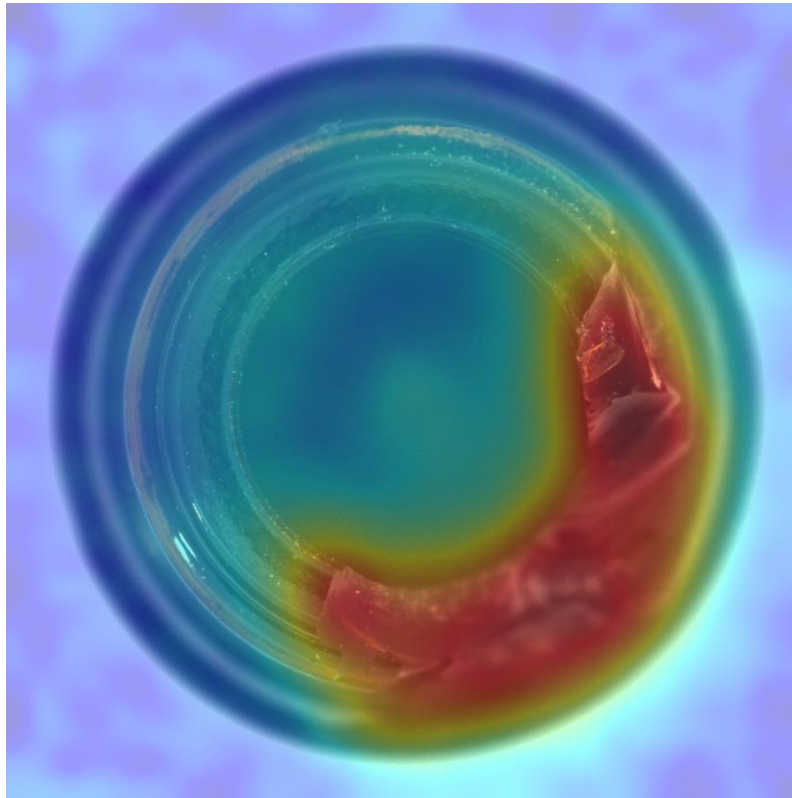


Fig 6.2: Generated anomaly heatmap for the defective sample — the highlighted (red/yellow) region correctly localizes the torn plastic wrap defect.



Fig 6.3: A second inference run on a known-normal image from the same category correctly returns a Normal verdict (score = 0.2052), confirming the model discriminates correctly between classes rather than flagging every input as defective.

Test Image	Expected	Predicted	Score	Result
bottle/test/broken_large/000.png	Defective	Defective	0.9399	Correct
bottle/test/good/000.png	Normal	Normal	0.2052	Correct

7. Observations

- The inference module correctly loads any of the 15 trained checkpoints and produces a clear, high-confidence separation between normal and defective samples (0.94 vs. 0.21 on the tested bottle example).
- Running inference on GPU (RTX 3050) is fast enough for single-image, on-demand use, unlike the CPU-bound coresnet selection step encountered during training.
- The generated heatmap correctly localizes the defect region visually, matching the same area highlighted in the Milestone 2 evaluation figures.
- The FastAPI wrapper exposes this functionality over HTTP, making it usable by a frontend or another backend service without requiring direct command-line access.
- Severity thresholds are currently a reasonable starting point based on limited manual testing and are expected to be tuned further once validated across more samples per category.

8. Conclusion

Milestone 3 turned the 15 PatchCore models trained in Milestone 2 into an actually usable system: a standalone inference script, an HTTP API for serving predictions, and a batch validation tool to confirm correctness across all trained categories. Combined with a reorganized repository structure, this establishes a working bridge between the trained models and any frontend or backend service that needs to call them for real-time defect inspection.